Distributed Student Management & Taskflow System - Full Project Documentation

1. Project Overview

This project is a full-stack distributed web application designed using a microservices architecture. It integrates multiple independent services including a Taskflow system (API Gateway and UI), a Student Management Service, and a Library Management Service. The system demonstrates modern software engineering principles such as separation of concerns, scalability, containerization, and service-oriented architecture.

2. System Architecture

The system is structured into multiple services that communicate over HTTP and are orchestrated using Docker Compose. - Taskflow Service: Acts as the API Gateway and frontend renderer using EJS templates. - Student Service: Handles all student-related CRUD operations and MongoDB interactions. - Library Service: Manages library resources, books, and related operations. - MongoDB: Central database container used by all services. - Nginx Reverse Proxy: Routes external requests to appropriate services. This architecture ensures modularity and allows independent scaling and deployment of services.

3. Technology Stack

The project uses the following technologies: Backend: - Node.js (Express.js framework) - MongoDB (NoSQL database) - Mongoose ODM Frontend: - HTML5, CSS3, JavaScript - EJS templating engine DevOps & Infrastructure: - Docker - Docker Compose - Nginx Reverse Proxy - WSL2 (Windows development environment) Logging & Monitoring: - Winston logging library - Morgan HTTP request logger

4. Microservices Design

Each service is independently developed and deployed: Student Service: Responsible for managing student data including registration, updates, deletion, and retrieval. It exposes RESTful APIs such as /students and /courses. Library Service: Handles library operations such as book management and resource tracking. Taskflow Service: Acts as the central user interface and API gateway. It routes requests to appropriate services using HTTP proxy middleware. This separation allows each service to evolve independently without affecting others.

5. API Gateway Implementation

The Taskflow system functions as an API Gateway. It uses HTTP proxy middleware to route incoming requests to appropriate backend services. For example: - /api/students → Student Service (Port 3001) - /api/library → Library Service (Port 3002) This design hides internal service complexity from the client and improves security and scalability.

6. Database Design

MongoDB is used as the primary database system. Each service has its own schema definitions: Student Schema: - name (String) - email (String, unique) - course (String) - enrollmentDate (Date) - status (active/inactive) Course Schema: - name - description - duration - status The database is containerized using Docker to ensure consistency across environments.

7. Dockerization Strategy

The system is fully containerized using Docker. Each microservice has its own Dockerfile: - Taskflow Dockerfile runs the API gateway and frontend - Student Dockerfile runs student service - Library Dockerfile runs library service Docker Compose is used to orchestrate all services together, including MongoDB and Nginx.

8. Nginx Reverse Proxy

Nginx is used as a reverse proxy to manage incoming traffic. It routes requests based on URL paths: - / → Taskflow frontend - /api/students → Student service - /api/library → Library service This improves performance, security, and simplifies client-side communication.

9. System Workflow

When a user accesses the system: 1. User visits the Taskflow UI. 2. Taskflow acts as a gateway. 3. Requests are routed to microservices. 4. Microservices interact with MongoDB. 5. Responses are returned to the UI. This workflow ensures a clean separation between frontend and backend services.

10. Security & Logging

Winston is used for structured logging across services. Morgan logs HTTP requests. Security considerations include: - Environment variables for secrets - Separation of services - API gateway abstraction Future improvements may include JWT authentication and role-based access control.

11. Challenges Faced

Key challenges included: - Docker engine configuration issues on Windows - Microservice communication setup - Route mismatches between frontend and backend - Database connection stability - Deployment orchestration These were resolved through structured debugging and architectural redesign.

12. Future Improvements

Future enhancements: - Kubernetes deployment for scaling - CI/CD pipeline using GitHub Actions - Authentication system (JWT/OAuth) - Role-based dashboards - Cloud deployment (AWS / Azure / Render)

13. Conclusion

This project demonstrates a modern distributed system using microservices architecture. It highlights skills in backend development, frontend integration, containerization, and system design. The modular structure ensures scalability, maintainability, and real-world production readiness.